WHY RUNES - the harness is the craft

A Ruby agent harness with a fabric, an identity, a memory and a face

Runes 0.4.0 | 11 November 2026 | github.com/La-Voix-du-chat-artiste/runes

A Ruby agent harness with a fabric, an identity, a memory, and a face. Written for engineers who suspect that the future of coding belongs to whoever masters the harness - not whoever rents it.

Palette: `primary-dark` - ruby `#EF4D8E` - neon `#A8DC4F` - blue `#5A9CF0`

TL;DR

- **Runes runs fleets of LLM agents** over a swappable message fabric (in-process <-> MQTT 3.1.1 <-> MQTT 5), with A2A for discovery and MCP for tools.
- **Every workflow verb is a plugin.** `cmd`, `ruby`, `chat`, `agent`, `map`, `repeat`, `call` - seven runes, between 68 and 728 lines each, all replaceable, and a new one takes about thirty lines.
- **It runs Shopify Roast workflows unmodified.** The shipped example is Roast's README example, byte for byte, and the test suite reads that exact file.
- **It treats the boring parts as the product**: capability guard, signed envelopes, a durable journal, verify-and-resume, and a Rails observatory that watches the whole fleet.
- **Execution is deduped, not just distributed.** MQTT 5 shared subscriptions hand each prompt to one agent; `Runes::RequestLedger` makes a redelivery or a retry a no-op that still gets an answer - and a refusal is a published event the observatory can count, not a line in a log.
- **The semantics of the seven runes are a frozen contract** - declare / execute / query, memoized execution, bodies that declare their input, control by verdict. The implementation can move (interpreted Ruby today, Spinel-AOT tomorrow, Nim agents at the edge); the contract does not.
- **A declarative fleet layer - world + rules - is specified** (`docs/FLEET_DSL.md`): agents, channels, routes, facts declared statically; rules that react to events; the guard and the broker ACL derived from the same file a human reads.
- **15 319 lines of `lib`, 11 773 lines of tests, 636 runs, 2 879 assertions, zero network in the suite.** Ruby 4.0.4, Rails 8.1.3.1, SQLite, mosquitto on localhost.

---

1. The future of coding is not writing code - it is conducting it

For sixty years the craft advanced by raising the level of abstraction: machine code, assembly, structured code, objects, garbage collection, managed runtimes, frameworks. Each step moved the programmer further from the metal and closer to intent. The agentic era is the next step, and it is a bigger one than all the others, because this time the machine does not just execute the program - it *writes parts of the program while it runs*.

That inverts the discipline. When the code is produced at runtime by something non-deterministic, the skills that made you senior - memorizing APIs, writing idiomatic loops, knowing the framework - depreciate. What appreciates is everything around the generated code: **the fabric that moves it, the identity that signs it, the**

guard that confines it, the journal that remembers it, and the language that describes what should happen. In one word: the harness.

The harness is the new runtime. The people who master it will run fleets the way an earlier generation ran servers - as owned, audited, repairable infrastructure. The people who rent it will be tenants of someone else's dashboard, unable to say what their agents did, on whose authority, and with what evidence. Runes is built so that you are in the first group.

2. A DSL is the right abstraction - and its semantics are the asset

Everyone is building agent frameworks. Almost nobody is asking what the *programming model* for a swarm should be. A swarm is not a function call graph - it is a **world** with inhabitants, places, routes, and rules that fire when the world changes. That is the insight Inform 7 proved and the one Runes keeps, minus the natural-language parsing: declare the world, declare the reactions, let the fabric carry the events.

On top of that world sits the workflow DSL, and here Runes is opinionated about what matters. Not the syntax - syntax is replaceable. The **semantics of the seven runes**, which are frozen as a contract:

Axiom	Statement
1. Declaration is not execution	`rune(:x)` declares; `rune!(:x)` executes; `rune?(:x)` asks
2. Execution is memoized	a step runs once; replay returns the recorded outcome
3. The body declares its input	a step returns what it consumes — the data graph is explicit and readable by a human, an agent, or a Rails app
4. Each rune has a type of output	`out/err/status` for processes, `value` for pure blocks, `response/session` for LLM calls, `iteration(i)` for loops, opaque subroutines for `call`
5. Control is by verdict	`skip!`, `next!`, `break!`, `fail!` — no hidden control flow

These five axioms are why a Roast file runs unmodified on Runes, why the observatory can draw any run as a timeline without guessing, and why the whole engine can be ported without breaking user code. Master the semantics and every implementation detail becomes negotiable.

3. From workflows to worlds: the fleet layer

A workflow says *how to compute a result*. A fleet says *who exists, who may touch what, and what happens when the world changes*. Runes 0.4 specifies that second layer (`docs/FLEET_DSL.md`):

```
fleet "prospection" do
transport :mqtt5

agent :writer do
model "glm-5.3-flash"
tools fs_write: :allow          # default-deny, always
end

route :scraper => :writer => :reviewer

on :contact_qualified do |e|
next! unless e.score > 0.7
task :writer, "Redige l'email pour %{name}"
end

on :guard_denied do |e|
notify "Refus: #{e.agent} / #{e.tool}", level: :warn
end
end
```

Three properties make this more than sugar:

- **Statically analyzable.** The file is a restricted Ruby subset -

a whitelist, enforced by an AST walker before evaluation. No `instance_eval` of foreign code, no metaprogramming, no hidden I/O. The loader fails closed: a fleet file that does not fully analyze leaves no subscriptions and no grants behind.

- **The guard sees it.** Tool grants, routes and rule actions are

extracted at load time into policy fragments and per-role mosquitto ACLs. The gap "the guard does not see runes" closes for the declarative layer by construction.

- **AOT-ready.** The same restricted subset is chosen to be compilable

by Spinel - the AOT Ruby compiler - so a fleet file that loads in the harness is a candidate for native single-binary deployment with no rewrite.

This is the Inform 7 lesson applied with discipline: the power of *world + rules*, the readability of a document, the auditability of a configuration, and the escape hatch of a real language underneath.

4. Eleven things that make this genuinely fun to build

1. Deleting the clever part made it stronger. 0.3 removed the entire claim/lease consensus protocol - about 250 lines of "who gets this work?" bookkeeping - and replaced it with MQTT 5 shared subscriptions. The broker now does exactly-once distribution natively. We proved it live: 200 messages, two clients in one shared group, 100/100 each, no dupes, no losses. **The best distributed-systems code is the code you get to delete.**

2. A hand-rolled MQTT 5 client, and it's only 1 149 lines. CONNECT/ CONNACK property probing, PUBLISH properties, `$share` subscriptions, keepalive, reason codes, reconnect-and-resubscribe - one readable file with a 33-test codec suite behind it.

3. The transport is a seam, so the tests never touch a broker. One contract, three implementations: in-process hub, MQTT 3.1.1, MQTT 5. Exactly-once dispatch across two agents is asserted in milliseconds, with no broker, no network, no flake. Swapping the fabric is a constructor argument.

4. A rune is a plugin, and it takes about thirty lines. No special-casing: dropping a class in adds a verb - and it inherits the harness's furniture (journal, registry, observatory) because it is a plugin, not a special case.

5. We adopted another project's DSL, and made the docs unfakeable. Runes runs Roast workflows unmodified, matched in both directions. `examples/analyze_codebase.rb` *is* the Roast README example, and the compatibility test reads that file - the docs cannot drift from the artifact.

6. Reading a fleet is its own reward. The Rails observatory reconstructs every agent's life: cards, online/offline (including Last Will), prompt lifecycles, tool calls, A2A handoffs, refusals over time. There is a specific delight in watching agents pass work to each other on a bus you built - and in a board a program can read (`GET /board.mmd`), because a board an agent cannot read is a screenshot.

7. The tests are hermetic, and that's a craft. Provider HTTP is faked with a real `Net::HTTPResponse` over a dup transport; the agent CLI is replaced at a factory seam; broker tests spawn their own broker on a free port. **636 runs, 2 879 assertions, no API key, no network, no broker** - the entire harness runs on a plane.

8. Ruby is the right language for this, and it's not close. Blocks with `instance_exec`, `method_missing` delegation, `define_singleton_method` binding each verb's `X`/`X!`/`X?` triple, `Struct` value objects, `Mutex` + `Thread` where Python would need a framework. And the future is brighter than that: Spinel compiles Ruby whole-program to a native binary - single-file deployment for every daemon, with the DSL semantics untouched.

9. The honest bits are fun too. `parallel` is threads, so a running iteration cannot be cancelled - written down, and tested. Known gaps are first-class in `STATE.md`. Nothing builds trust faster than a project that tells you where it is weak.

10. The fleet got a face - and then a voice. Every packet carries a signature verdict and the key's fingerprint; one `agent_id` under two keys is a finding. The guard publishes its denials so the page draws refusals over time instead of pointing at a log nobody reads.

11. At-least-once is a bug you can make boring. `Runes::RequestLedger` (187 lines): claim the `request_id` before any work is queued, remember what the first copy did, answer a replay from that. Bounded, TTL'd, one TTL, atomic - sixteen threads, one winner. A whole genre of distributed-systems anxiety collapses into a hash with a clock.

5. Why this is more important than it looks

Agents are becoming infrastructure, and infrastructure has requirements. A prompt loop is a demo. The moment an agent can write files, run commands, spend money and talk to other agents, it needs identity, least privilege, audit, observability. Runes builds the guard, the journal, the signatures and the observatory as the product - the model call is the easy part.

Non-determinism makes auditability a correctness property. You cannot reproduce an LLM run by re-reading the code. So every prompt lifecycle is appended to a durable, rotating, `flock`-protected journal and published on the bus; `bin/runes-replay` shows or replays it. **Your agents' memory should live on your disk, in a format you can read - not in a vendor's dashboard.**

"It can do anything" is the threat model. Default-deny per tool *and* per action on the resolved path; tool RPC off unless enabled and secret-gated with constant-time compare; path confinement; WASM isolation with a fuel budget for untrusted tools; broker-level fail-closed ACLs. Layers, not vibes.

On a shared bus, "unsigned" means "unknown". Ed25519 signed envelopes with a per-agent trust store turn `agent_id` from a claim into a fact. Nothing else in the ecosystem treats the message bus as an adversarial surface.

Interop, or you're an island. A2A for discovery, MCP for tools, swappable transport, pluggable providers, Roast-compatible DSL. Adopting Runes does not lock you in - and that is the point.

The alternative to a SaaS is not a smaller SaaS - it is a file. Most of what a small team rents is an *engine*: idea -> plan -> run -> verify -> tell someone what to do next -> keep the receipts. That engine fits in one readable workflow (`examples/prospect_pipeline.rb`, 463 lines) writing files a human, an agent and a Rails app can all read. The parts worth paying for - identity, tenancy, money, law - stay in an app; the intelligence becomes malleable, in your repo, runnable by any harness, and it inherits the safety properties a cron job never had.

Mastering the harness is the moat. Frameworks age, providers merge, models get cheaper. What compounds is your command of the layer that outlives them all: the language you describe work in, the fabric that moves it, the proofs it left behind. Runes is an attempt to make that layer beautiful enough to master.

6. Receipts

`lib/`	**15 319 lines** across 65 files
`test/`	**11 773 lines** across 48 files
Suite	**636 runs, 2 879 assertions, 0 failures** — no keys, no provider calls, no broker
Observatory	Rails 8.1 app — fleet, runs, topology, traces, who published, what was refused, all through `Runes::Transport`
Executables	**7**: `runes`, `runes-daemon`, `runes-client`, `runes-mcp`, `runes-replay`, `runes-acl`, `runes-workflow`
The seven runes	`agent` 728, `chat` 503, `repeat` 202, `cmd` 205, `map` 183, `ruby` 90, `call` 68
MQTT 5 adapter	**1 149 lines**, hand-rolled, live-verified against mosquitto 2.1.2 — and it *reconnects*
Security surfaces	**1 716 lines** — guard, envelopes, identities, trust store, telemetry, RPC auth
Request ledger	**187 lines** — the execution half of exactly-once

CRM pipeline	**463 lines** of DSL writing a real pipeline into interoperable files
Live proof	**200 messages, one shared group, 100/100 split, no dupes, no losses**

Every number is measured, not remembered - `ruby scripts/receipts.rb` prints all of them. This file is also the source of `docs/WHY_RUNES.pdf`, so a stale number here is a one-command bug.

7. What we're honest about

- Distribution is exactly-once; execution is deduped in-process. The remaining seam is stated: a request with no `request_id` has no identity to dedupe on; the ledger is in-process; a durable ledger is the next step.
- Workflow runes ask permission only when asked to (`RUNES_WORKFLOW_POLICY`), and its patterns match command text exactly - it wants a narrow allowlist, not a wildcard.
- Token scanning is not a sandbox. Real confinement needs `sandbox-exec`/`bwrap`.
- A2A peer cards are unauthenticated (discovery-only).
- `parallel` is threads: no cooperative cancellation of a running iteration.
- MQTT 3.1.1 is compatibility-only - no shared groups, no properties. It refuses loudly.
- The observatory has no auth - fine on localhost, a prerequisite for anything exposed.
- The observatory reaches the harness by load path, not gem dependency - a deliberate coupling with a sharp edge.
- One-shot tool feedback: a single build plan does not yet loop plan->execute->feed-back. Missions compensate with verify-and-resume.

Every one of those is a `git grep` away from a doc and a test.

8. Where this goes next

Shipped since the last roadmap: identity badges, guard telemetry, broker-free observer ingest, the bounded request ledger.

What is actually left, in order:

- **The fleet layer** (`docs/FLEET_DSL.md`): restricted-subset loader with Prism AST walker, static policy/ACL extraction, conformance levels L1/L2 in 0.4.0.
- **A plugin catalogue** (`/plugins`) generated from `Runes::Plugin.names` - the "a rune is a plugin" claim as a page.
- **Alerts** - impersonation and refusals get a row an operator can acknowledge.
- **The plan chain** on one page: prompt -> plan -> tool calls -> evidence -> verifier verdict.
- **Runs, diffed and replayed** - "re-run just the step that failed".

- **An observatory that is an agent** - its own A2A card and MCP server, so any agent can ask *"what happened on the bus?"* mid-task.
- **Retention and rollups at real volume**, and **a flight recorder** for offline replay.
- **The Spinel conformance gate** - CI compiles fleet files and workflow matchers under Spinel, non-blocking while the compiler matures.

The paragraph to steal

> Runes is a Ruby agent harness built on one belief: the future of coding belongs to whoever masters the harness. A swappable message fabric (in-process, MQTT 3.1.1, MQTT 5 shared subscriptions), A2A discovery, MCP tools, Ed25519-signed envelopes, a default-deny capability guard, a durable journal with verify-and-resume, and a Rails observatory that watches the whole fleet - with the unglamorous half of agent infrastructure treated as the product. Its workflow DSL is Roast-compatible and its seven verbs are plugins; their semantics - declare, execute once, declare your input, control by verdict - are a frozen contract that outlives any implementation. A declarative fleet layer (world + rules) sits on top, statically analyzable, guard-visible, AOT-ready. Fifteen thousand lines of library, eleven thousand lines of hermetic tests, no network required, no vendor in the path. The alternative to a SaaS is not a smaller SaaS - it is a file, and this harness keeps it yours.